Classes e Objetos - Implementação

Orientação a Objetos – DCC025

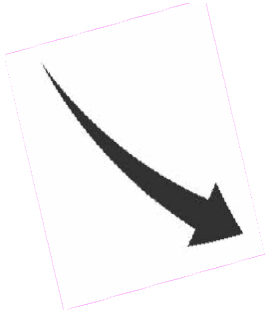
Profª. Bárbara de Melo Quintela
barbara@ice.ufjf.b

Universidade Federal de Juiz de Fora – UFJF
Departamento de Ciência da Computação - DCC

2012.1

Implementação de Classes

```
[public] class <Identificador>
{
        <corpo da classe>
}
```



```
1  public class Pessoa {
2
3      private String nome;
4
5      public String getNome(){
6          return nome;
7      }
8
9      public void setNome(String nome){
10         this.nome = nome;
11     }
12     ...
18 }
```

Implementação de Classes

```
[public] class <Identificador>
{
    <corpo da classe>
}
```

```
1  public class Pessoa {
2
3      private String nome;
4
5      public String getNome(){
6          return nome;
7      }
8
9      public void setNome(String nome){
10         this.nome = nome;
11     }
12     ...
18 }
```

Classes e Método Principal

```
1  public class Pessoa {
2
3      private String nome;
4
5      public String getNome(){
6          return nome;
7      }
8
9      public void setNome(String nome){
10         this.nome = nome;
11     }
12
13     public static void main(String args[]){
14
15         Pessoa p1 = new Pessoa();
16         System.out.println(p1.getNome());
17     }
```

Método main() dentro da classe Pessoa

Classes e Método Principal

```java
public class Pessoa {

    private String nome;

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        nome = nome;
    }
```

```java
public class Principal {

    public static void main(String args[]){

        Pessoa p1 = new Pessoa();
        System.out.println(p1.getNome());
    }
```

Prefira construir uma classe de execução, que, quando necessário, irá chamar os métodos da classe Pessoa

Programação Java - Classe

```
<modificador> <tipo> <identificador>(<parâmetros>)
{
        <corpo do método>
}
```

- Métodos
  - Métodos que pertencem ao objeto são chamados de métodos de instância. Analisando a definição padrão de métodos, temos:
    - <modificador> é um modificador de acesso
    - <tipo> é o tipo do valor de retorno do método
    - <identificador> é o identificador do método
    - <parâmetros> é uma lista de parâmetros.
  - Além disso, o corpo do método é composto de comandos, expressões e declarações de variáveis locais ao método.

Métodos

```
1  public class Pessoa {
2
3      private String nome;
4
5      public String getNome(){
6          return nome;
7      }
8
9      public void setNome(String nome){
10         this.nome = nome;
11     }
12     ...
18 }
```

```
<modificador> <tipo> <identificador>(<parâmetros>)
{
        <corpo do método>
}
```

Modificadores - Introdução

Modificadores em Classes

```
[public] class <Identificador>
{
        <corpo da classe>
}
```

- Palavra reservada public
  - É opcional e indica que a classe pode ser referenciada por qualquer outra classe além do próprio pacote. Se a declaração da classe não for precedida de public, então ela só poderá ser referenciada por outras classes do mesmo pacote (visibilidade default)
    - OBS: Os modificadores de acesso private e protected não podem ser aplicados às classes.
  - O <Identificador> é usado para referenciar a classe.
  - No <corpo da classe> são definidos os atributos e métodos da classe.

Modificadores em Classes

```
1  public class Pessoa {
2
3      private String nome;
4
5      public String getNome(){
6          return nome;
7      }
8
9      public void setNome(String nome){
10         this.nome = nome;
11     }
12     ...
18 }
```

Regra Geral:

Defnir como "private" todos os atributos da classe

Regra Geral:

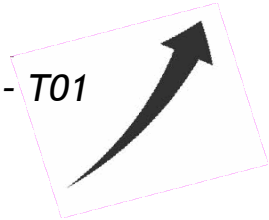
Definir como público os métodos da classes. OBS: alguns métodos deverão, em circunstância específicas, ser definidos como privados. Contudo, como geral geral, utilize "public"

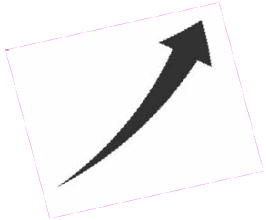
Exemplo de Acesso

```
public class Aluno {

    public int matricula;
    public Date dataNascimento;
    public String nome;

}
```

```
public class Aluno {

    private int matricula;
    private Date dataNascimento;
    private String nome;

}
```

Tentativa de acesso 01 - T01





Tentativa de acesso 02 - T02

```
public class Principal {

    public static void main(String args[]){

        Aluno aluno01 = new Aluno();

    }
}
```

Modificadores

Modificador	*Descrição*
`default`	Somente classes do mesmo package possuem acesso
`public`	Todos possuem acesso
`protected`	Apenas os membros da classe e subclasse
`private`	Apenas os membros da classe

Modificador	*Descrição*
`static`	A variável ou método é comum a todas as instâncias da classe.
`final`	O valor da variável não pode ser modificado.

Modificadores

Static

Um atributo declarado com o modificador static significa que é um atributo da classe e não de instância. Em outras palavras, existirá apenas um atributo, ocupando uma única posição de memória, em vez de um atributo para cada instância, cada um com sua própria posição de memória.

Como resultado, o atributo existirá antes mesmo que qualquer objeto seja criado

Outro efeito é que o atributo será compartilhado por todas as instâncias da classe. Desta forma, se um objeto alterar o conteúdo do atributo, a alteração afetará todos os outros objetos da classe.

Modificadores

Static

O modificador static também pode ser usado na declaração de métodos. Neste caso o método fica ligado à classe e não à instância, sendo denominado de método de classe. Como consequência o método pode ser acessado antes de existir um objeto da classe.

- Exemplo de método static: o método main()

Exemplo - Static

```
3  public class Livro {
4
5      private String nome;
6      private double preco;
7      private double desconto;
8
9      public double getDesconto() {
10         return desconto;
11     }
12
13     public void setDesconto(double desconto) {
14         this.desconto = desconto;
15     }
16 }
```

Exemplo – Static

```
3  public class Livraria {
4
5      ...
6
7      public double calculaPrecoFinal(Livro book1) {
8
9          double valor = book1.getPreco() * (1.0 - book1.getDesconto() );
10         return valor;
11     }
12
13     ...
14
15 }
```

=

Se o livreiro tiver a política de descontos únicos para toda a loja, pode ser que o atributo desconto tenha alguma inconsistência, algum valor diferente de um livro para o outro - o que seria indesejável.

Como garantir que todos os livros tenham o mesmo desconto ?

Exemplo – Static

```
3  public class Livro {
4
5      private String nome;
6      private double preco;
7      private static double desconto;
8
9      public static double getDesconto(){
10         return desconto;
11     }
12
13     public static void setDesconto(double d ){
14             desconto = d;
15     }
16 }
```

Modificadores - Final

Final

O modificador final pode ser usado em atributos, métodos e, também, classes. Impede que se modifique o que está sendo pré-fixado.

Final aplicado sobre:

- Atributos: torna os mesmos constantes
- Métodos: impede sobrescrita de métodos
- Classes: impede a criação de classes filhas (impedir herança)

- OBS: Sobrescrita (sobrescrever): Métodos de assinaturas iguais (mesmo método em termos de assinatura, mas podem possuir implementações diferentes. Não confundir com sobrecarga (métodos de mesmo nome, mas assinaturas diferentes.

Modificadores - Final

```java
public final class MyClass {}

// Isso não é permitido
public class MyOtherClass extends MyClass {}
```

```java
public class MyClass {
    public final void foo() {}
}

public class MyOtherClass extends MyClass {
    public void foo() {} // Não é permitido sobrescrever o método
}
```

```java
public class MyClass {

    public void foo() {
        final int teste = 0;
        teste++; // Não é permitido
    }
}
```

Palavra Reservada “this”

Referência “this”

Em algumas situações é necessário referenciar o próprio objeto corrente. Por essa razão, todo objeto possui um atributo especial identificado por this, que é uma referência para o próprio objeto.

- O this é um apelido para o endereço de memória do objeto

Palavra Reservada “this”

```
public class objetoGeo
{
    protected Color cor;
    protected int x, y;

    public objetoGeo(Color cor, int x, int y)
    {
        this.cor = cor; this.x=x; this.y = y;
    }
    public Color retCor() {return cor};
}
```

Atributo

Variável

Este exemplo mostra um uso típico do atributo this. Ele mostra um método cujo parâmetro formal possui o mesmo nome de um atributo de instância. Para distinguí-los, é necessário qualificar o atributo da instância com o atributo this.

Teste

Teste do “this”

Crie uma classe qualquer (escolha um nome. Ex: Aluno)

Crie um atributo privado nome (String)

Crie um método

- Receba por parâmetro um nome (String)
- Atribua o valor recebido como parâmetro ao atributo nome
- Não faça uso do “this”

Verifique o resultado

Depois, altere o nome do parâmetro

Verifique o resultado

Depois, use o this no caso anterior e no primeiro caso

Demais Resultados

```
3  public class Aluno {
4
5      private String nome;
6
7      public void alteraNome(String valor){
8          nome = valor;
9      }
10
11 }
```

```
3  public class Aluno {
4
5      private String nome;
6
7      public void alteraNome(String valor){
8          this.nome = valor;
9      }
10
```

```
3  public class Aluno {
4
5      private String nome;
6
7      public void alteraNome(String nome){
8          this.nome = nome;
9      }
10
11 }
```

Exemplo implementação de Métodos

Exemplo

Classe que retorna máximo de três valores do tipo ponto flutuante:

```
import java.util.Scanner;
public class EncontraMaximo
{
    public void determinaMaximo()
    {
        //cria scanner para entrada a partir da janela de comando
        Scanner input = new Scanner(System.in);
        //solicita tres valores de ponto flutuante
        System.out.print("Informe tres numeros de ponto flutuante separados por espaços: ");
        double numero1 = input.nextDouble();//lê o primeiro double
        double numero2 = input.nextDouble();//lê o segundo double
        double numero3 = input.nextDouble();//lê o terceiro double
        //determina o valor maximo
        double result = maximo(numero1, numero2, numero3);
        //exibe o valor maximo
        System.out.println("O maximo é: " + result);
    }
```

Exemplo

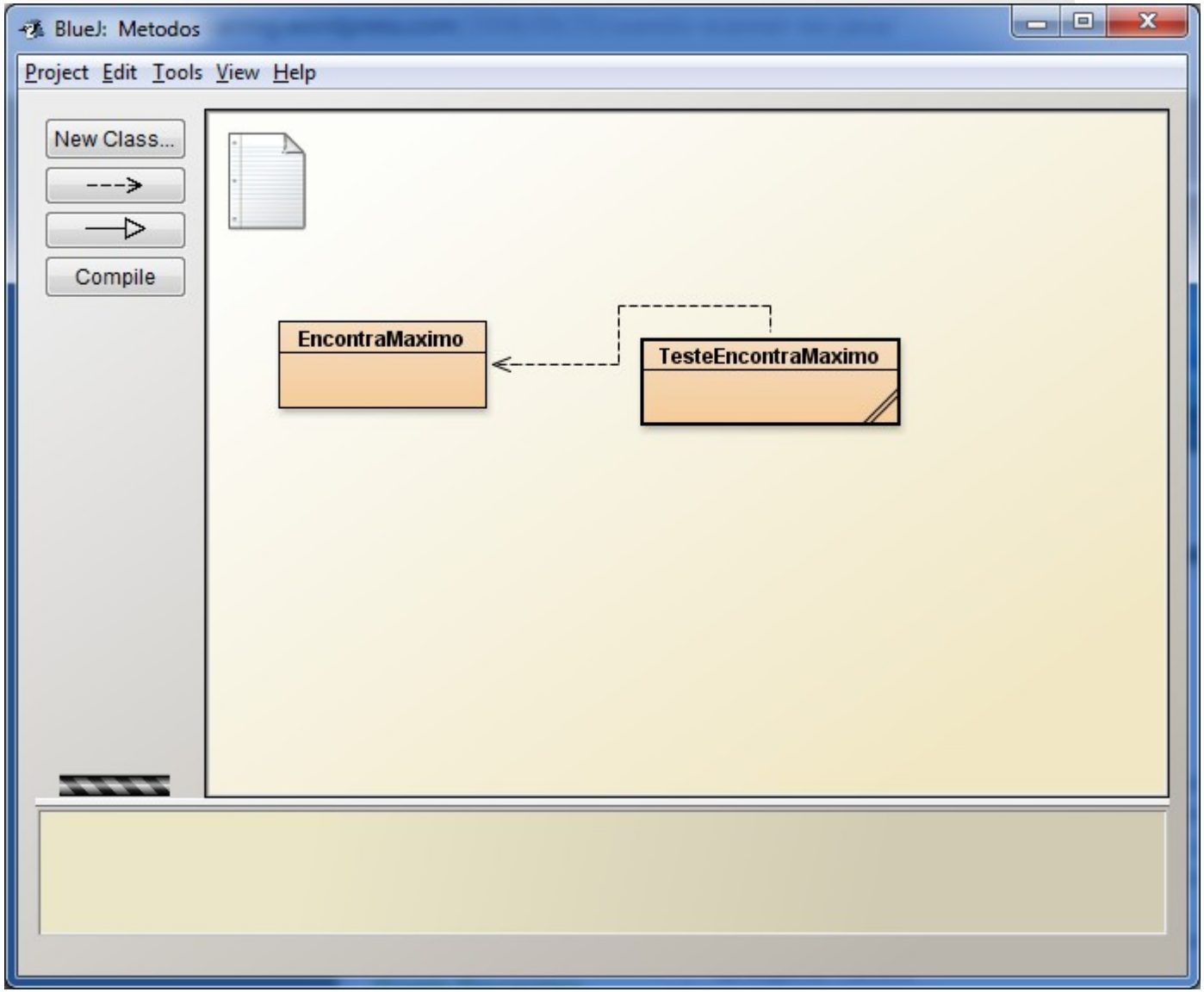
Método declarado pelo programador para retornar o valor máximo

```
public double maximo(double x, double y, double z)
{
    double valorMaximo = x; //supoe que x é o valor maximo
    //determina se y é maior que o valorMaximo
    if (y > valorMaximo)
        valorMaximo = y;
    //determina se z é maior que o valorMaximo
    if (z > valorMaximo)
        valorMaximo = z;
    return valorMaximo;
}
}
```

Exemplo

Testando

```
public class TesteEncontraMaximo
{
    public static void main (String[] args){

        EncontraMaximo encontraMaximo = new EncontraMaximo();
        encontraMaximo.determinaMaximo();
    }
}
```


BlueJ: Metodos
Project Edit Tools View Help
New Class...
Compile
EncontraMaximo
TesteEncontraMaximo

Exemplo

Podemos usar um método da classe Math:

```
public double maximo(double x, double y, double z)
{
    return Math.max(x, Math.max(y,z));
}
}
```

Exercício

- Exercício 01
  - Crie uma classe Empregado que inclui 3 campos como variáveis de instâncias – nome (tipo string), sobrenome (tipo String) e um salário mensal (tipo double). Forneça um método “set” e um “get” para cada campo.
  - Crie uma classe de teste. Instancie um objeto da classe Empregado e atribua os valores “Pedro”, “Silva” e 1000 para seus atributos. Imprima o nome do empregado e seu salário atual
  - Em seguida
    - Crie um método aumentarSalario(). Esse método deve ler o salário atual do empregado, dobrar esse valor e atualizar no salário do empregado. Depois, imprima o novo salário

Solução – Classe Empregado

```
3  public class Empregado {
4
5      private String nome;
6      private String sobrenome;
7      private double salario;
8
9      public String getNome() {
10         return nome;
11     }
12
13     public void setNome(String nome) {
14         this.nome = nome;
15     }
16
17     public String getSobrenome() {
18         return sobrenome;
19     }
20
21     public void setSobrenome(String sobrenome) {
22         this.sobrenome = sobrenome;
23     }
24
25     public double getSalario() {
26         return salario;
27     }
28
29     public void setSalario(double salario) {
30         this.salario = salario;
31     }
32 }
```

Solução – Classe de Execução

```
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Empregado p1 = new Empregado();
8
9          p1.setNome("Pedro");
10         p1.setSobrenome("Silva");
11         p1.setSalario(1000);
12
13         System.out.println("Nome Empregado: " + p1.getNome() + " " + p1.getSobrenome() );
14         System.out.println("Salário: R$" + p1.getSalario());
15     }
```

Solução – Novo Método

```
3  public class Empregado {
4
5      private String nome;
6      private String sobrenome;
7      private double salario;
8
9      public String getNome() {
10         return nome;
11     }
12
```

```
28
29     public void setSalario(double salario) {
30         this.salario = salario;
31     }
```

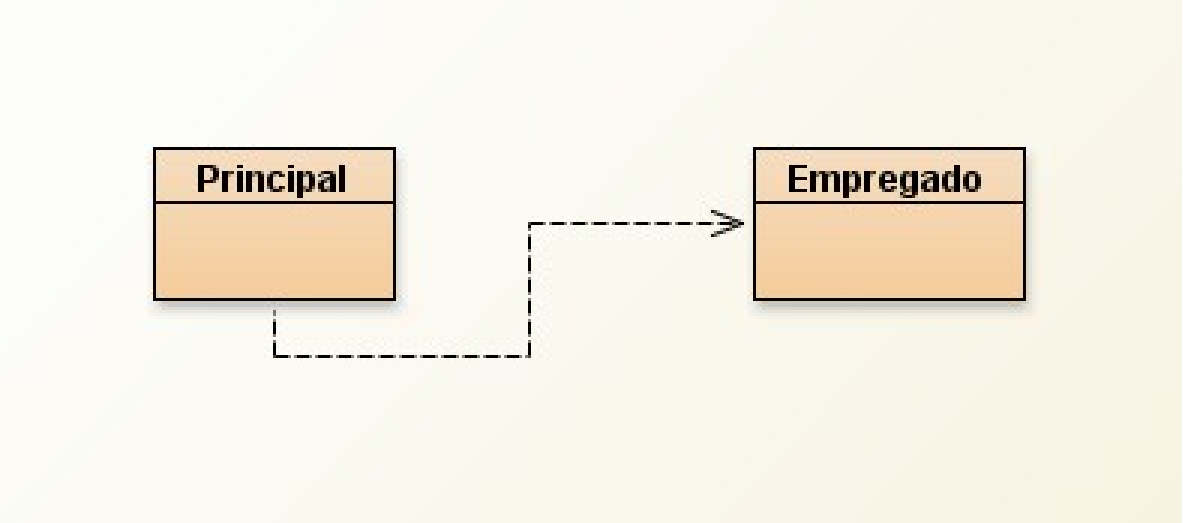
```
33     public void aumentarSalario(){
34         double salarioAtual = this.getSalario();
35         double novoSalario = salarioAtual * 2;
36         this.setSalario(novoSalario);
37     }
38 }
```

Solução – Novo Método

```
public void aumentarSalario(){
    double salarioAtual = salario;
    double novoSalario = salarioAtual * 2;
    salario = novoSalario;
}
```

```
public void aumentarSalario(){
    setSalario(salario * 2);
}
```

```
public void aumentarSalario(){
    setSalario(getSalario() * 2);
}
```

```
public void aumentarSalario(){
    salario = salario * 2;
}
```

Representação em Modelo